

Capstone Project Usage Instructions. Read it carefully.

- The project/assessment will be available for the full project/assessment window.
- You may **start, pause, and continue** your assessment anytime during this window by using the **Save & Close** button.
- When you click **Save & Close**, your progress will be saved and you can return later to continue the assessment.
- After clicking **Save & Close**, if you log in again immediately, the **Resume** button may appear **disabled for about 7–8 minutes**.
 - This is normal; please wait a few minutes for the Resume button to become active.
- Do **not** click **Finish Test** until you are completely done with your project.
- Once you click **Finish Test**, your assessment will be **submitted permanently**, and you will **not** be able to log in or make any further changes.

Capstone Project – Cloud Native Microservices Engineering

1. Introduction

The Capstone Project focuses on developing a cloud-native microservices application named **Cake Delight**. The project is intended to provide hands-on experience in designing, developing, containerizing, deploying, and managing independently deployable services in a cloud-native environment.

Through this project, participants will learn by building a practical application that reflects real-world engineering concerns such as service ownership, loose coupling, scalability, fault tolerance, observability, and deployment automation.

2. Problem Statement

Design and develop a cloud-native application named **Cake Delight** using a microservices-based architecture. The application should deliver an end-to-end customer journey that allows users to browse cake products, apply filters, add items to a basket, modify basket contents, complete checkout, rate purchased cakes, and receive order confirmation notifications.

The solution must demonstrate cloud-native engineering principles including independent service deployment, API-based communication, event-driven notifications, containerized execution, database-backed persistence, and Kubernetes-based orchestration.

3. Project Objectives

By completing this capstone project, participants should be able to:

- Implement business capabilities using a microservices architecture.
- Design service boundaries for catalog, order, rating, and notification capabilities.
- Enable service-to-service communication through REST APIs and/or messaging.
- Use containerization to package and run application services consistently.
- Deploy and manage services using Kubernetes.
- Apply scalability, resilience, and maintainability practices in application design.

4. Technology Stack

Participants may use the following technology stack to implement the solution:

- **Microservices Framework:** Spring Boot or Node.js
- **Containerization:** Docker
- **Orchestration:** Kubernetes
- **Communication:** REST APIs and/or message broker

- **Supporting Components:** API Gateway, database services, and notification mechanism
- **Core Services:** Cake Catalog, Order, Rating, and Notification

5.Functional Scope

The Cake Delight application should support the following functional capabilities:

- Browse available cakes in the catalog.
- Filter cakes by attributes such as name, category, and price range.
- Add selected cakes to the shopping basket.
- View, update, and remove items from the basket.
- Complete checkout and create an order.
- Submit ratings for cake items.
- Send order confirmation notifications after successful checkout.

6. Microservices Design

6.1 Cake Catalog Microservice

- Maintain cake product information such as name, description, category, price, availability, and image reference.
- Provide APIs to list cakes and retrieve details of a selected cake.
- Support filtering by product name, category, and price range.

6.2 Order Microservice

- Manage basket operations such as adding, updating, and removing cake items.
- Display basket contents and calculate order totals.
- Create orders during checkout and maintain order status.

- Publish an order completion event after successful checkout.

6.3 Rating Microservice

- Allow users to submit ratings for cake items.
- Store and retrieve ratings for individual products.
- Expose APIs to calculate and display average ratings.

6.4 Notification Microservice

- Listen for order completion events from the Order Microservice.
- Send order confirmation through email, SMS, or in-app notification.
- Maintain notification delivery status where applicable.

7. Architecture Overview

The solution should follow a microservices architecture where the API Gateway acts as the single entry point for client requests. Each microservice should own its business capability and data persistence mechanism. Services may communicate synchronously using APIs and asynchronously using events or messages.

Key Architecture Components:

- Client application or user interface
- API Gateway
- Cake Catalog Microservice
- Order Microservice
- Rating Microservice
- Notification Microservice
- Databases owned by respective services
- Message broker for event-driven communication

Architecture Characteristics:

- Each service should be independently buildable, deployable, and scalable.
- Services should interact through clearly defined APIs or messaging contracts.
- The application should be containerized using Docker.
- Deployment, scaling, and service discovery should be handled through Kubernetes.
- The design should consider fault tolerance, retries, logging, and basic monitoring.

8. Expected Deliverables

- Source code for all implemented microservices.
- API documentation for exposed endpoints.
- Dockerfiles for each service.
- Kubernetes deployment and service configuration files.
- Database schema or data model for applicable services.
- Message/event contract for order completion notifications.
- Setup and execution instructions.
- Short demonstration of the end-to-end application flow.

9. Evaluation Criteria

Evaluation Area	Expected Focus
Microservices Design	Clear service boundaries, independent ownership, and loose coupling.
API Implementation	Well-defined endpoints, request/response handling, and validation.
Event Handling	Order completion event flow and notification processing.
Containerization	Correct Docker packaging and service execution.

Kubernetes Deployment	Deployment files, services, configuration, and scalability readiness.
Reliability and Maintainability	Error handling, logging, basic monitoring, and clean code structure.
End-to-End Demo	Successful demonstration of browsing, basket, checkout, rating, and notification flow.

10. Conclusion

The Cake Delight capstone project provides participants with practical exposure to building a real-world cloud-native microservices system. By completing this project, learners will demonstrate their ability to design scalable services, integrate APIs and events, containerize applications, deploy workloads on Kubernetes, and deliver an end-to-end business workflow.